DATAVAULT APPLICATION TESTING

Product understanding Document:

1. Product Overview

Application Name: DataVault
Type: Zero-Trust File Sharing Platform
Platforms: Android (Owner App) + Web Portal (Guest Access)

2. App Purpose

DataVault is a security-focused file-sharing platform designed to prevent static link leakage.

It ensures that shared files are accessible only under strict conditions:

- Time-based expiry
- View-based limitation
- Optional password protection
- Manual revocation (Kill Switch)
- Real-time access logging

The system follows a Zero-Trust security model where every access attempt is validated before file exposure.

3. Target Users

File Owners (Primary Users)

- Upload files
- Set expiry time
- Set view limits
- Add optional password
- Revoke links manually
- Monitor access analytics

Guests (Recipients)

- Receive portal link
- Enter password (if required)
- Access file under defined conditions

4. Core Features

- User Authentication (Firebase)
- Secure File Upload (Cloudinary integration)
- Custom Expiry Date & Time
- Custom View Limits
- Password Protection
- Web Portal Access Validation
- Access Analytics (logs & pie chart)
- Active Revocation (Kill Switch)
- Vault Dashboard with real-time status

5. Business-Critical Flows

1. Secure Share Flow
2. Password Shield Flow
3. Temporal Expiry Flow
4. View Limit Enforcement
5. Manual Revocation Flow
6. Vault Cleanup (Delete File)

TEST PLAN

DataVault_TestPlan_v1.0

1. Scope of Testing

In Scope

- User Registration & Login
- Session persistence
- File upload process
- Expiry time logic
- View limit logic
- Password validation
- Web portal validation
- Firestore real-time synchronization
- Manual revocation
- File deletion
- Analytics logging
- URL parameter parsing

Out of Scope

- Firebase internal infrastructure
- Cloudinary internal backend
- OS-level vulnerabilities
- Third-party server downtime

2. Features to Test

- Authentication flow
- Secure upload parameters
- Access control logic (isAccessAllowed)
- Expiry countdown
- View count increment logic
- Password protection mechanism
- Revoke button functionality
- Delete file functionality
- Portal validation process
- Analytics logging accuracy
- State synchronization between Android and Web

3. Features Not to Test

- Internal Firebase server performance
- Cloudinary storage architecture
- Network provider reliability
- Device hardware limitations

4. Test Types

Functional Testing

Validate all business flows and logic.

UI Testing

Verify layout consistency, dark mode readability, responsiveness.

API Testing

Test Firebase authentication and Cloudinary REST integration.

Regression Testing

Re-test critical flows after bug fixes.

Security Testing

Test URL tampering, brute force attempts, unauthorized access.

Boundary & Negative Testing

Test edge cases and invalid inputs.

Concurrency Testing

Test simultaneous access attempts.

5. Tools Used

- Manual Testing
- Postman (API validation)
- Chrome DevTools (Network inspection)
- Firebase Console
- Android Emulator / Physical Android Device
- GitHub Issues (Bug Tracking) /jira

6. Test Environment

Android

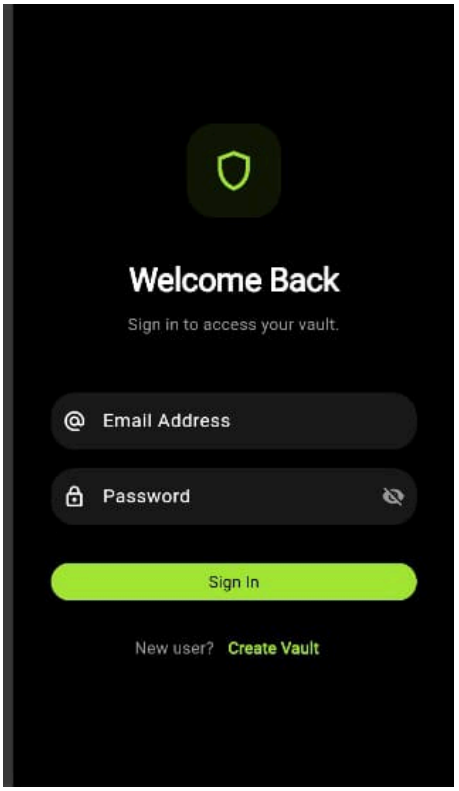
- Android 12+
- Physical device (Samsung / Xiaomi etc.)
- Emulator (API 31+)

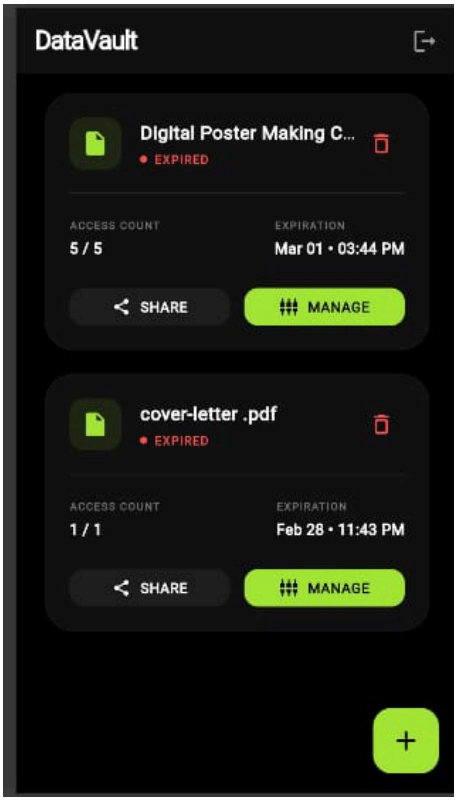
Web

- Google Chrome
- Microsoft Edge
- Desktop & Mobile view

Backend

- Firebase Authentication
- Cloud Firestore
- Cloudinary REST API





TEST SCENARIOS

ID	Scenario
TS-01	Verify user can register with valid credentials
TS-02	Verify login fails with invalid password
TS-03	Verify session persists after app restart
TS-04	Verify file uploads with valid parameters
TS-05	Verify upload fails without file selection
TS-06	Verify expiry time blocks access after defined time
TS-07	Verify view limit blocks access after limit reached
TS-08	Verify password-protected file blocks incorrect password
TS-09	Verify revoke button blocks portal instantly
TS-10	Verify deleted file returns 404 on portal
TS-11	Verify currentViews increments only after successful download
TS-12	Verify portal blocks invalid ?id parameter

TS-13	Verify analytics logs successful access
TS-14	Verify analytics logs failed password attempt
TS-15	Verify concurrent access does not bypass view limit
TS-16	Verify dashboard updates in real-time
TS-17	Verify expired files show correct status in dashboard
TS-18	Verify revoked files show blocked state
TS-19	Verify countdown timer accuracy
TS-20	Verify system blocks access when file is manually deleted

Each test case is referenced in its scenario for traceability.

MANUALLY TESTING TESTCASES AND MARKING STATUS

FUNCTIONAL TESTING

Test Case 1 - TS-01

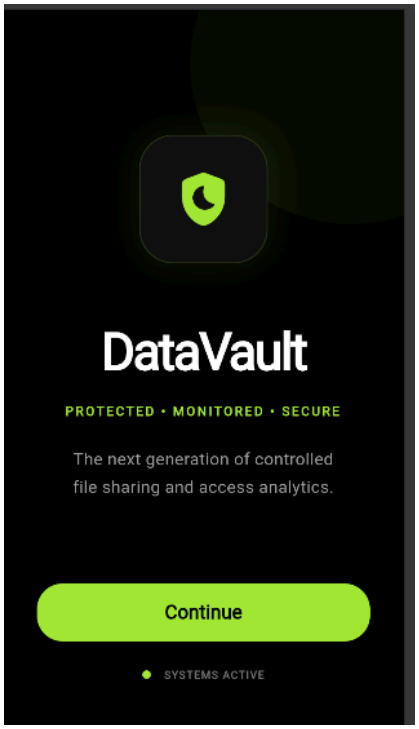
Test Case ID: TC-AUTH-01
Title: Verify login with valid credentials
Precondition: User account already exists

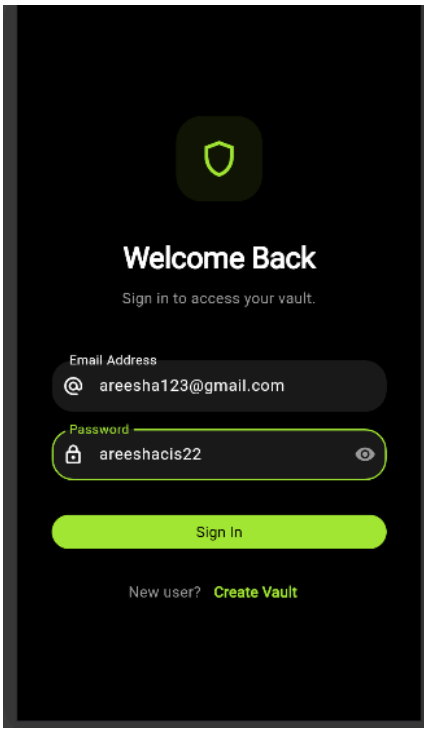
Steps:

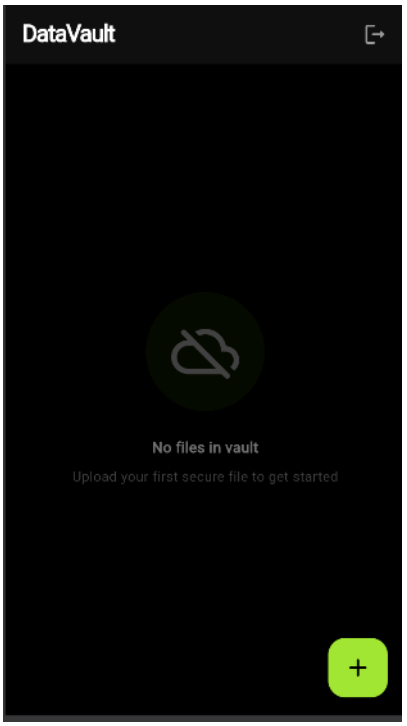
1. Open DataVault app
2. Enter valid registered email
3. Enter correct password
4. Click Login

Expected Result:
The user is successfully redirected to Dashboard.

Status = Passed







Test Case 2 - TS-02

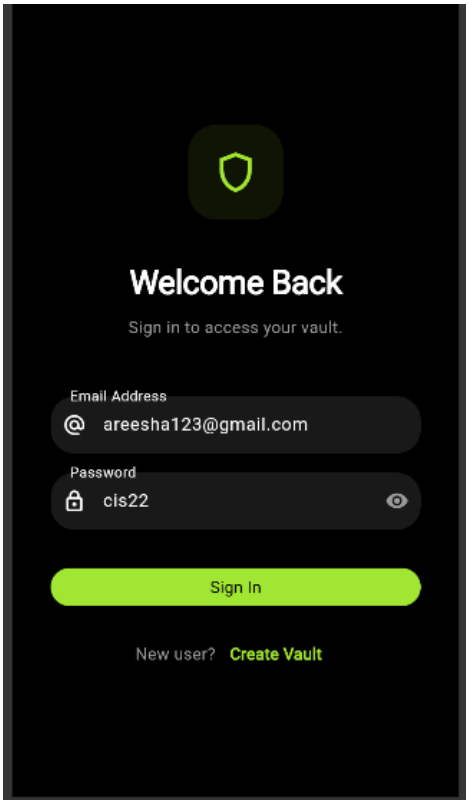
Test Case ID: TC-AUTH-02
Title: Verify login with invalid password
Precondition: User account exists

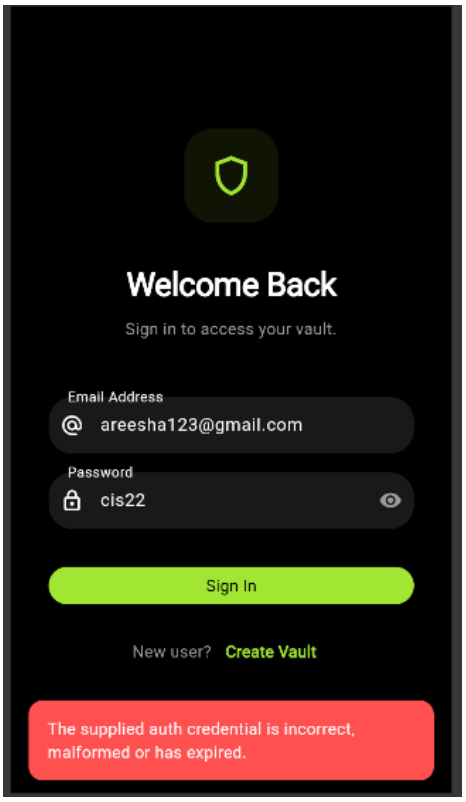
Steps:

1. Open app
2. Enter valid email
3. Enter incorrect password
4. Click Login

Expected Result:
System shows error message: Invalid Credentials.
The user remains on the login screen.

Status : Passed





Test Case 3 - TS-04

Test Case ID: TC-UPL-01
Title: Verify file upload with valid parameters

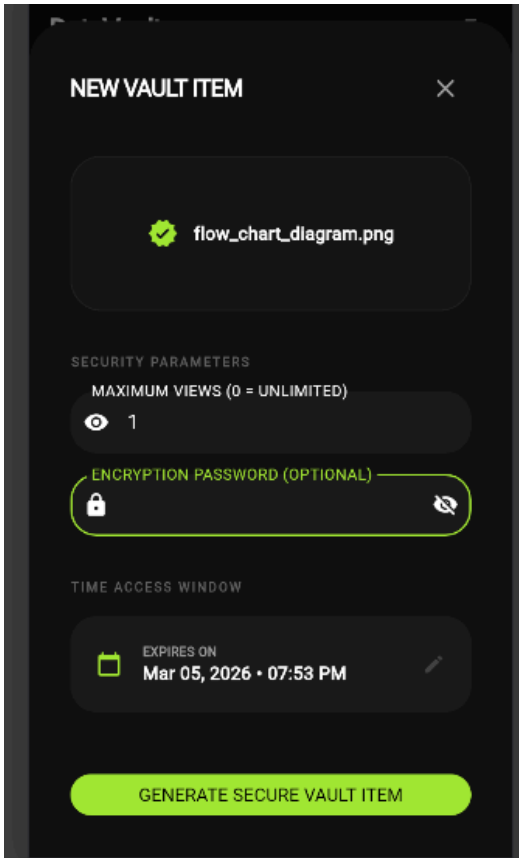
Precondition: User logged in

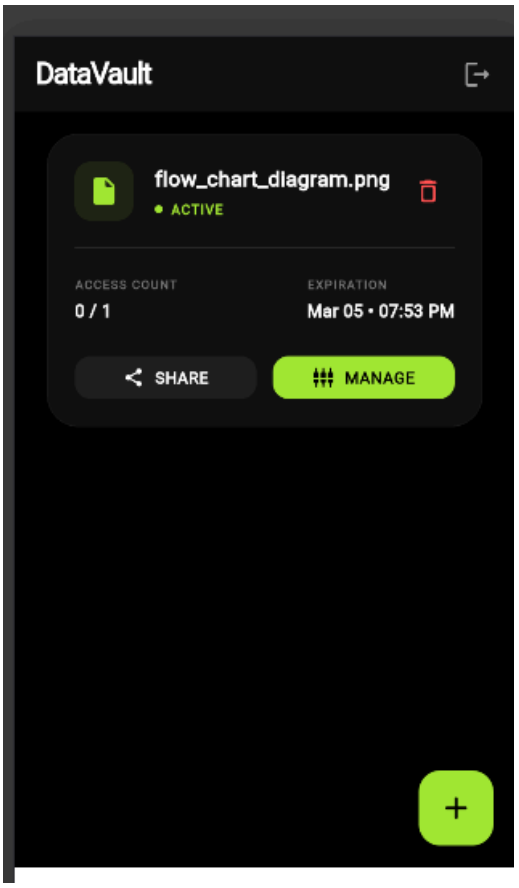
Steps:

1. Click Upload (icon)
2. Select file
3. Set expiry = 10 minutes
4. Set view limit = 1
5. Leave password empty(as it is optional to add)
6. Click Submit

Expected Result:
File uploads successfully.
Entry appears in the dashboard with status Active.

Status = Passed





Test Case 4 - TS-06

Test Case ID: TC-LOGIC-01
Title: Verify view limit enforcement

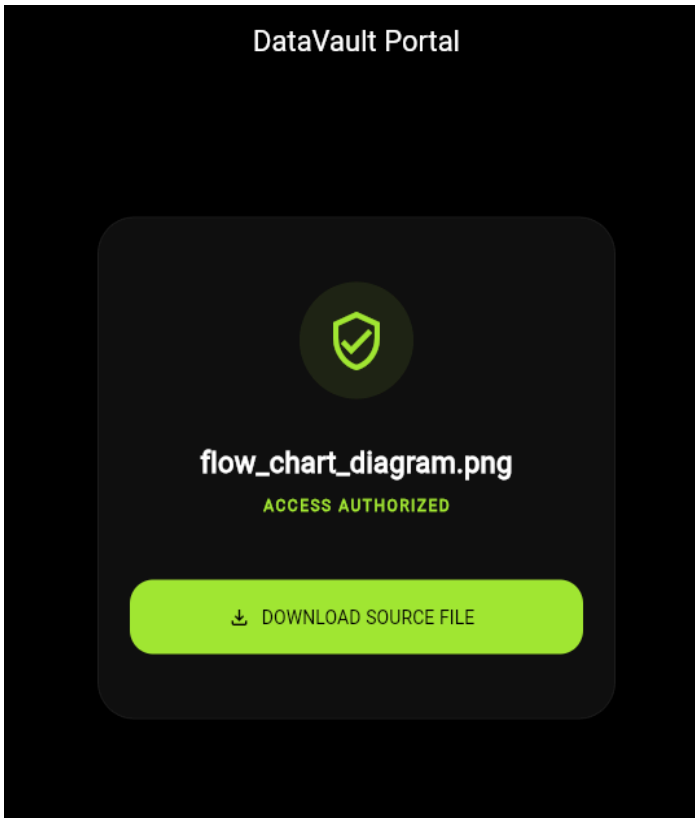
Precondition: File uploaded with view limit = 1

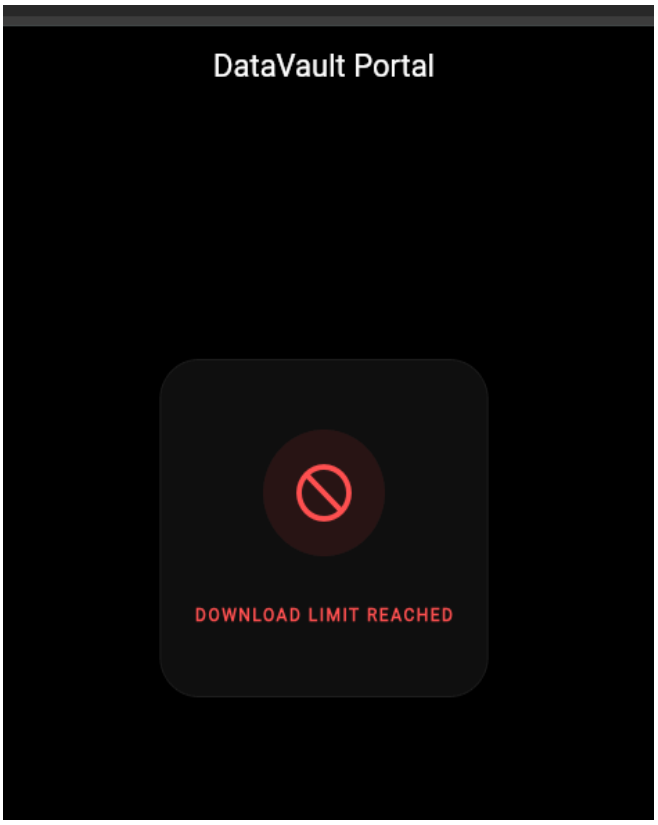
Steps:

1. Open portal link
2. Download file
3. Refresh page

Expected Result:
The first download succeeds.
The second attempt shows Access Blocked.
currentViews = 1 and limit reached

Status = passed





Test Case 5 - TS-012

Test Case ID: TC-SEC-01
Title: Verify portal blocks invalid URL parameter

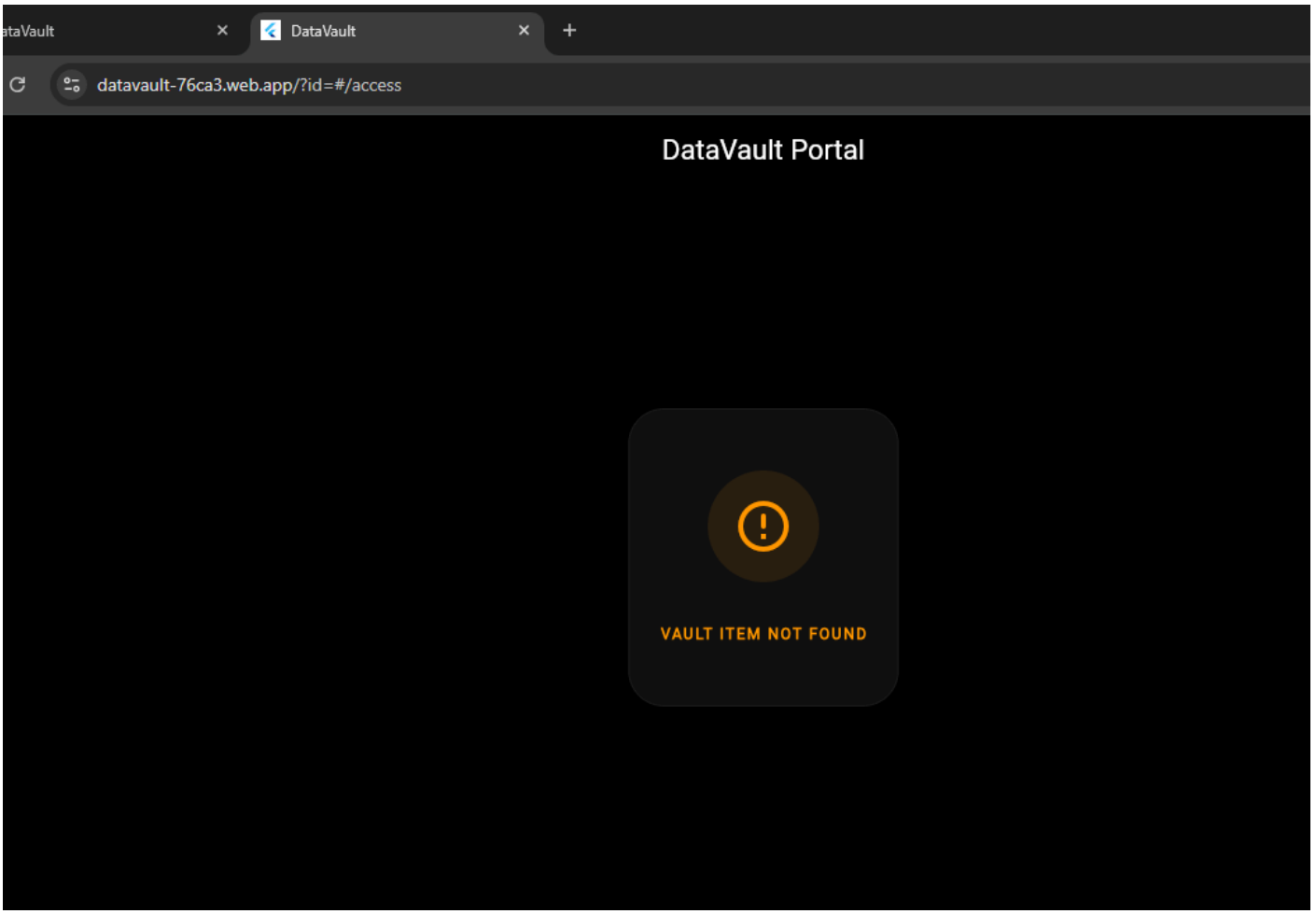
Precondition: Portal accessible

Steps:

1. Remove ?id parameter from URL
2. Press Enter

Expected Result:
Portal displays Invalid Link message vault item not found.
No file data exposed.

Status Passed



Test Case 6 - TS-09

Test Case ID: TC-RV-01
Title: Verify manual revoke blocks access

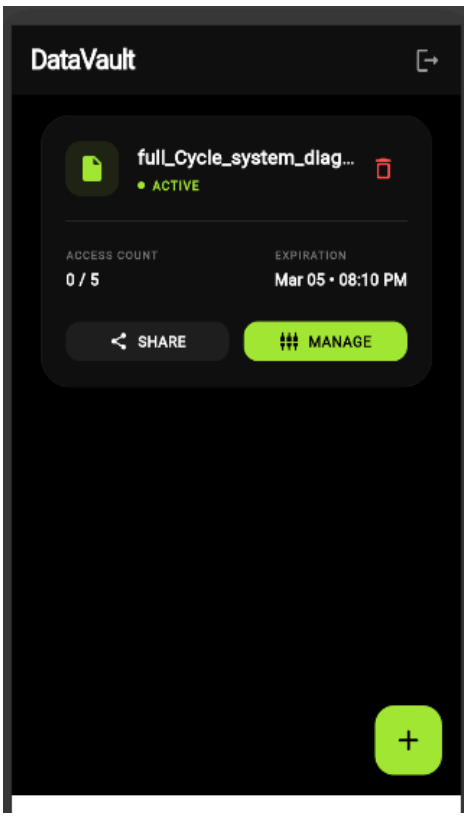
Precondition: File is Active

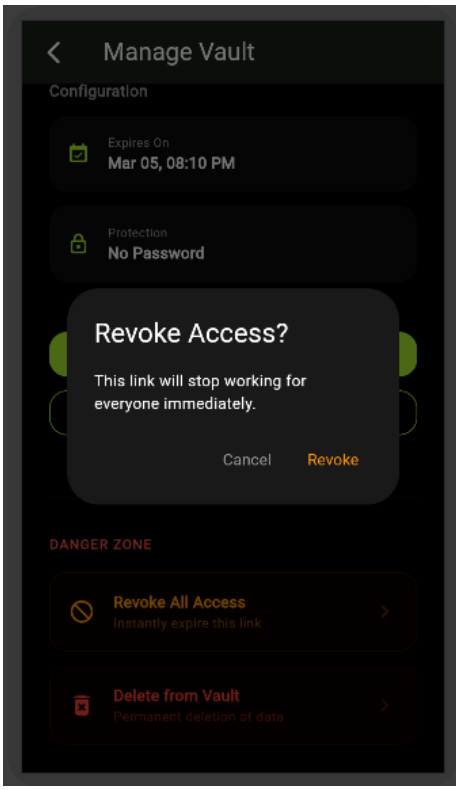
Steps:

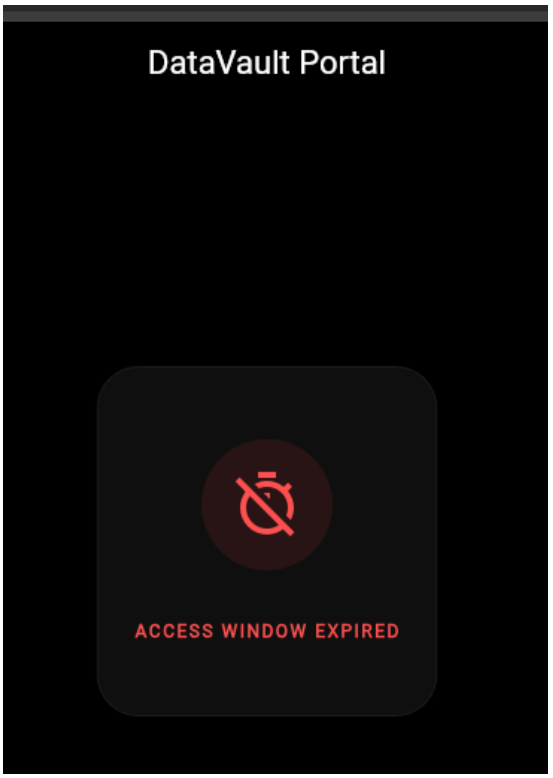
1. Open portal in browser
2. From dashboard click Revoke
3. Refresh portal

Expected Result:
Portal displays Access Blocked immediately.
Dashboard status updates to Revoked.

Status = Passed







Test Case 7 - TS-07

Test Case ID: TC-CON-01
Title: Verify concurrent access control

Precondition: View limit = 1

Steps:

1. Open link in two browsers
2. Click download simultaneously

Expected Result:
Only one download succeeds.
Second request blocked.

Status = Failed user can download file simultaneously with view limit =1

Test Case 8 - TS-08

Test Case ID: TC-CON-02
Title: Verify password-protected file blocks incorrect password

Precondition: file is uploaded successfully to dashboard and is protected by

password

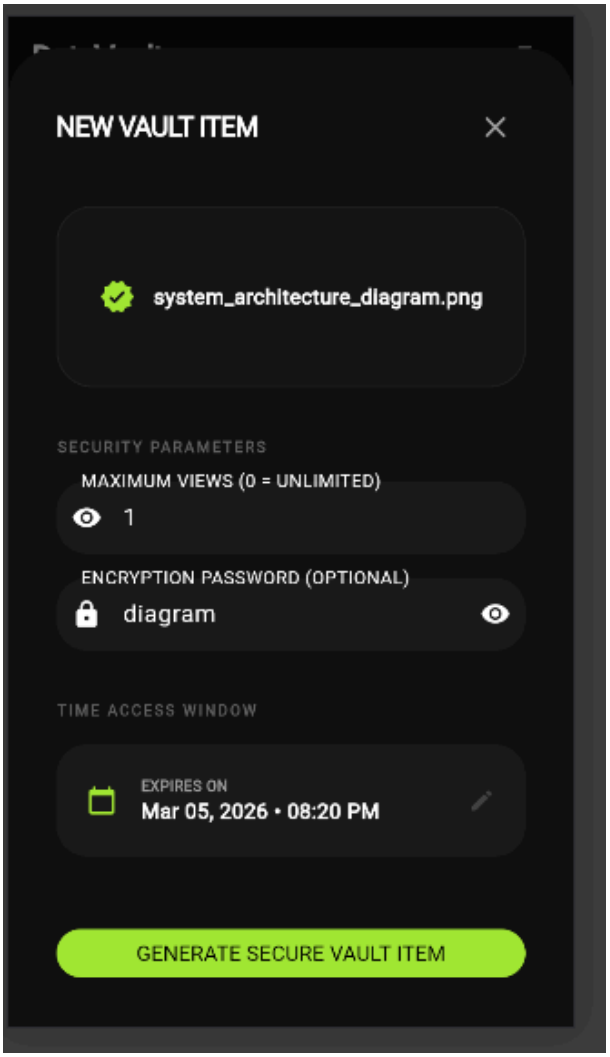
Steps:

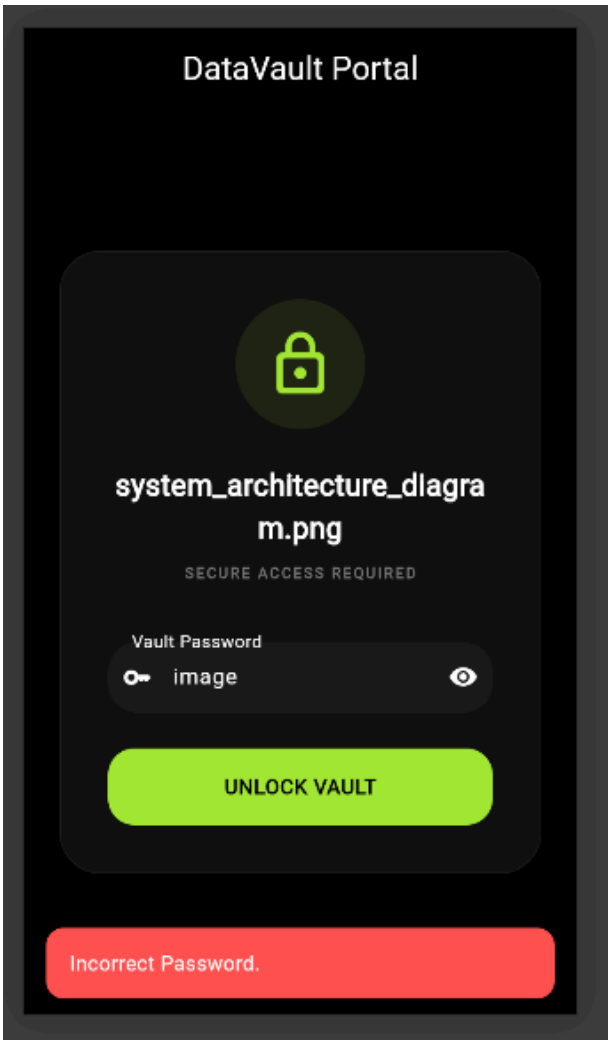
1. Click on link or open portal link in browser
2. Enter invalid password

Expected Result:
Access denied showed show up on screen and

file unable to be downloaded

Status = Passed





Test Case 9 - TS-013

Test Case ID: TC-CON-03
Title: Verify analytics logs successful access

Precondition: user successfully enter password and file is ready to download

Steps:

3. Click on manage button
4. Scroll down to see analytics log access section

Expected Result:

A successful authorize access log should be made and shown on screen

with date and time the attempt made and attempt increments by 1

Status = Passed

Test Case 10 - TS-014

Test Case ID: TC-CON-04
Title: Verify analytics logs failed password attempt

Precondition: user successfully directed to web portal

and enter a invalid password to access the file and access denied

Steps:

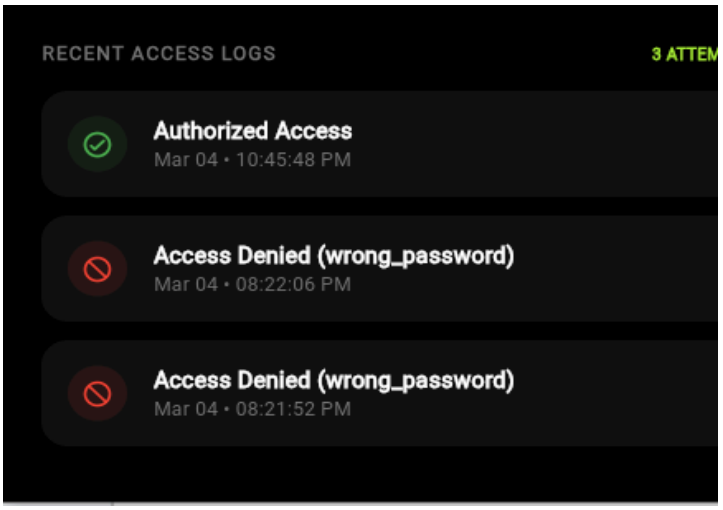
1. Click on manage button
2. Scroll down to see analytics log access section

Expected Result:

A access denied (wrong password) log should be made and shown on screen

with time and date the attempt made and attempt should not increment or increment by 0

Status= Passed



TESTCASES (NON-FUNCTIONAL)

Performance Testing

Test Case 1 – App Loading Time

- **Test Case ID:** TC-NF-01
- **Scenario Reference:** TS-01 / TS-04
- **Title:** Verify app launches within acceptable time
- **Precondition:** App installed, no previous sessions active
- **Steps:**
    1. Open the DataVault app
    2. Measure time taken to reach the Dashboard
- **Expected Result:** App loads in <2 seconds

Status = Passed

Test Case 2 – File Upload Performance

- **Test Case ID:** TC-NF-02
- **Scenario Reference:** TS-04
- **Title:** Verify large file upload performance
- **Precondition:** User logged in
- **Steps:**
    1. Upload multiple large files (50–100 MB each)
    2. Observe upload time and app behavior
- **Expected Result:** Files upload successfully without crash; upload time < acceptable threshold (e.g., 10 sec per file)

Status = Failed user were not able to upload large files above 20MB

Test Case 3 – Scrolling Performance

- **Test Case ID:** TC-NF-03
- **Scenario Reference:** TS-16
- **Title:** Verify smooth scrolling in Dashboard / Analytics
- **Precondition:** Multiple files exist in dashboard
- **Steps:**
    1. Scroll through dashboard and analytics lists rapidly
- **Expected Result:** No lag, flickering, or freezing
  Status = Passed

Security Testing

Test Case 4 – Unauthorized File Access

- **Test Case ID:** TC-NF-05
- **Scenario Reference:** TS-07 / TS-08
- **Title:** Verify user cannot access other users' files
- **Precondition:** Two separate accounts exist
- **Steps:**
    1. Login with User A
    2. Try accessing User B's portal link directly
- **Expected Result:** Access denied; error message shown
  Status = Passed

Test Case 6 – API Security

- **Test Case ID:** TC-NF-06
- **Scenario Reference:** TS-04 / TS-08
- **Title:** Verify API rejects invalid/expired tokens
- **Precondition:** API endpoint accessible
- **Steps:**
    1. Send API request with invalid token
    2. Send API request with expired token
- **Expected Result:** API returns 401 Unauthorized; no data exposure

  Status = Passed

Usability Testing

Test Case 7 – Navigation Clarity

- **Test Case ID:** TC-NF-07
- **Scenario Reference:** TS-01 / TS-04
- **Title:** Verify user can perform basic actions without guidance
- **Precondition:** First-time user
- **Steps:**
    1. Open app
    2. Try uploading a file and sharing link
- **Expected Result:** User completes tasks within 1–2 minutes without guidance

Status = Failed , layout/UI should be optimized for better user experience

Test Case 8 – Error Message Clarity

- **Test Case ID:** TC-NF-08
- **Scenario Reference:** TS-02 / TS-08
- **Title:** Verify all error messages are clear and understandable
- **Precondition:** User attempts invalid actions
- **Steps:**
    1. Enter wrong password
    2. Leave required fields blank
    3. Attempt upload without file
- **Expected Result:** Error messages clearly indicate the problem and corrective action

Status = Passed

UI/UX & Compatibility Testing

Test Case 9 – Responsive Layout

- **Test Case ID:** TC-NF-09
- **Scenario Reference:** TS-16 / TS-19
- **Title:** Verify UI works on multiple screen sizes
- **Precondition:** App installed on mobile device and emulator
- **Steps:**
    1. Open app on small screen (phone)
    2. Open app on large screen (tablet / emulator)
- **Expected Result:** Layout adapts; no cut-off or misaligned elements

Status = Passed

Bug Reports

Report 1 – Functional Defect

Bug ID: BUG-FUNC-01

Related Test Case: TC-CON-01
Scenario: TS-07 – Concurrent Access Control **Priority:** High
Module: File Download / Access Control
Environment: Android (mention version), Web (if applicable), Flutter App

Title

View limit restriction not enforced – Multiple users can download file simultaneously despite limit = 1

Description

When the file view/download limit is set to **1**, multiple users are still able to download the file simultaneously.
The system fails to enforce concurrency control.

Steps to Reproduce

1. Set file view limit to 1
2. Open the file link in two different browsers/devices
3. Click download at the same time

Expected Result

Only one download should succeed.
The second request should be blocked with a proper error message (e.g., "View limit exceeded").

Actual Result

Both users are able to download the file simultaneously.

Impact

- Business rule violation

Status

Open

‘

Report 2 – Non-Functional Defect (Performance)

Bug ID: BUG-NF-01

Related Test Case: TC-NF-02
Scenario: TS-04
Severity: Medium **Priority:** Medium
Module: File Upload
Environment: Flutter App (Android),

Title

Application fails to upload files larger than 20MB

Description

Application is unable to upload files above 20MB.
Large file upload (50–100MB) either fails or does not complete successfully.

Steps to Reproduce

1. Login to application
2. Attempt to upload file larger than 20MB
3. Observe upload behavior

Expected Result

The system should upload large files successfully without crashing.
Upload time should be within acceptable performance threshold.

Actual Result

Files above 20MB fail to upload.

Impact

- Limits usability for enterprise users
- Performance limitation

Status

Open

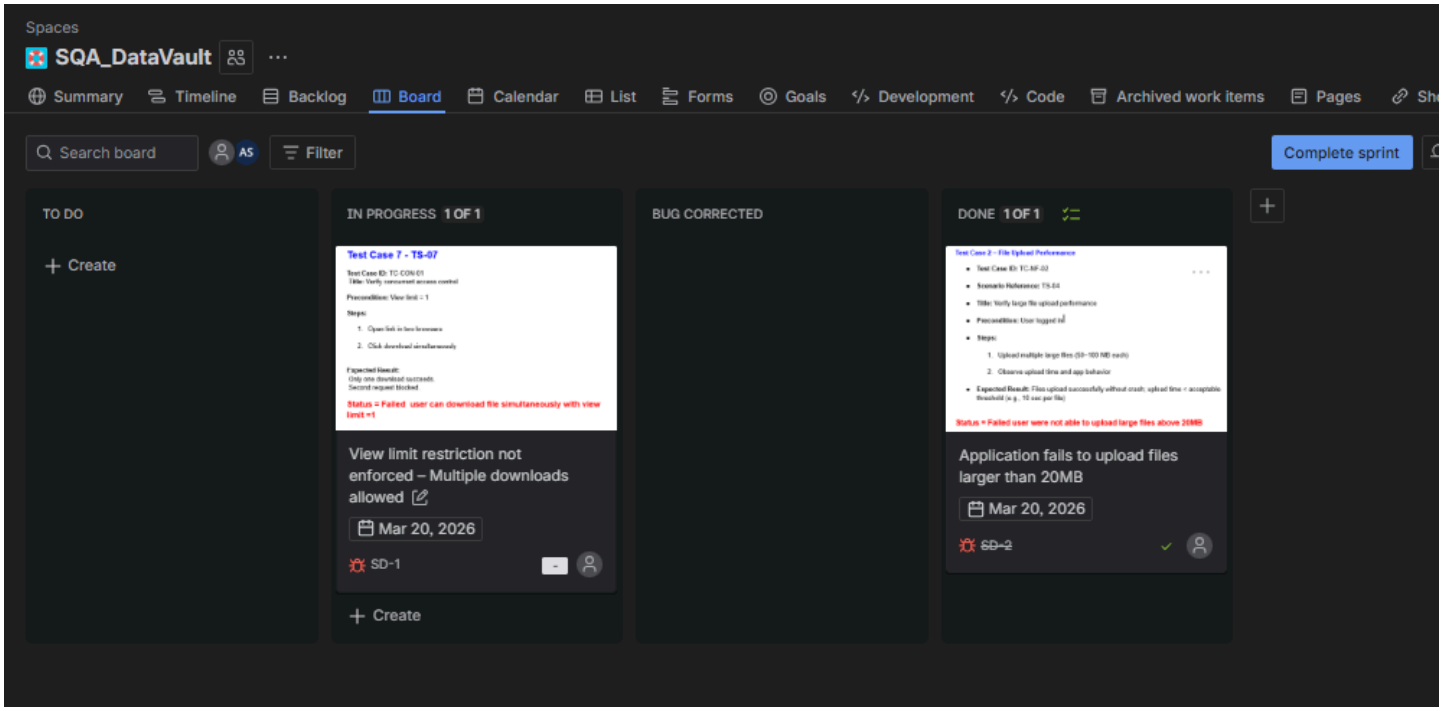
Testing Summary

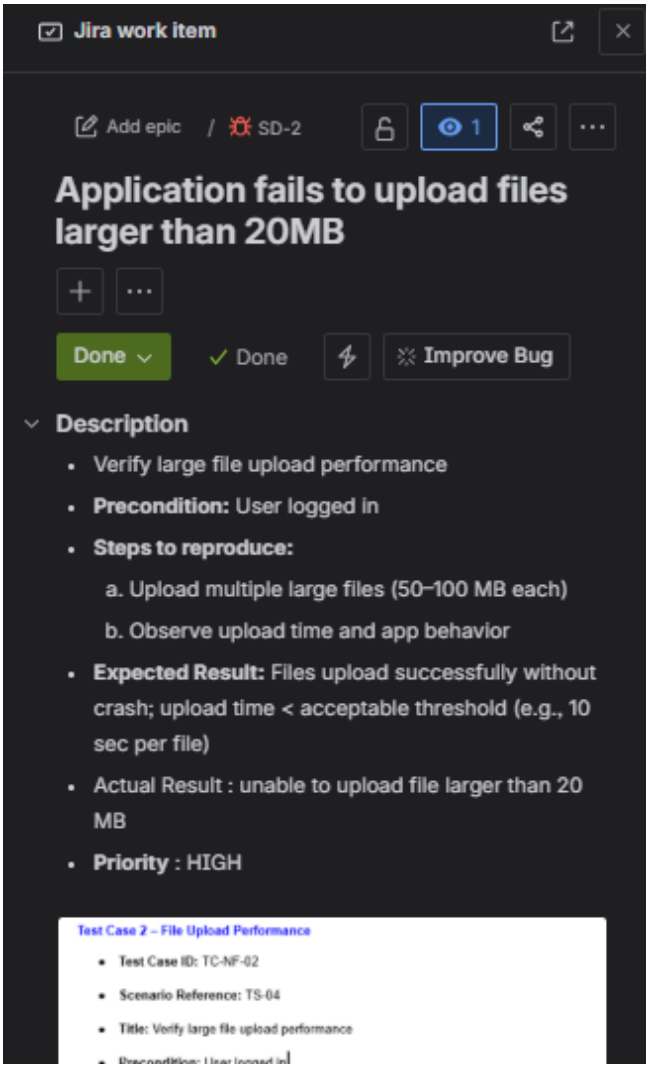
Total Test Cases Executed: 18
Passed: 16
Failed: 02
Blocked: 02

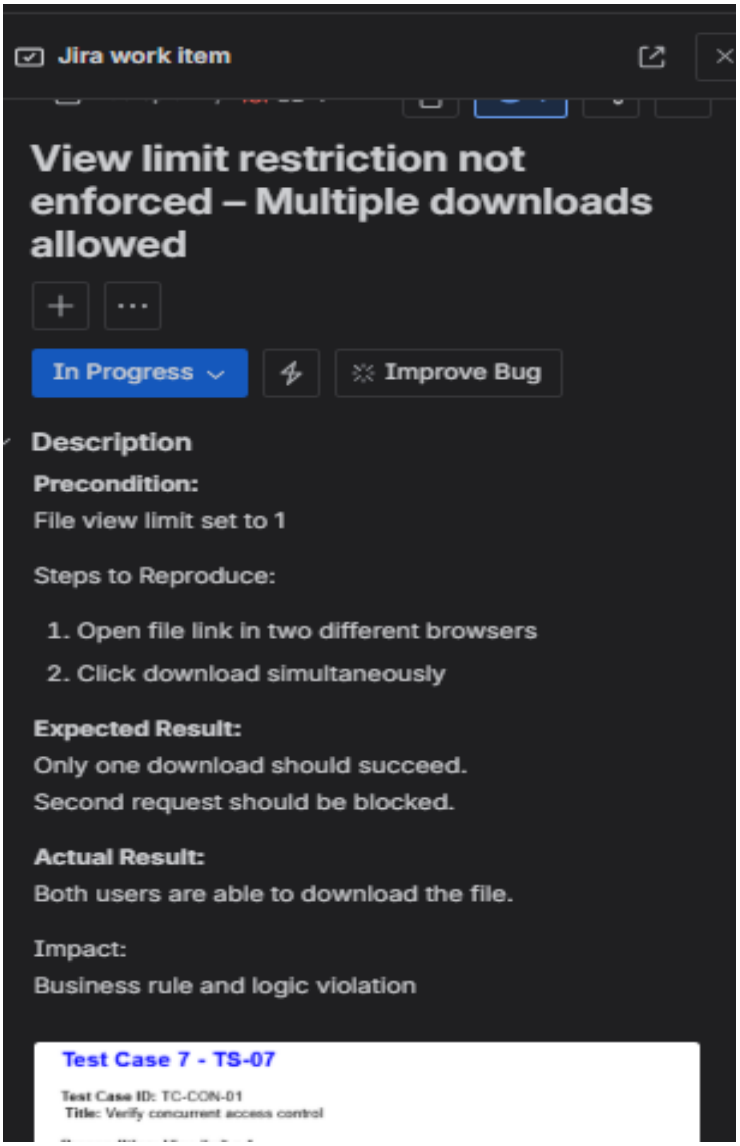
Key Defects Identified:

- Concurrency control failure (functional)
- Large file upload limitation (non functional)

BUG REPORTING IN JIRA







API testing Via Postman

Base endpoint: https://api.cloudinary.com/v1_1/dnb4odmqv/auto/upload

It sends file to cloudinary server , we will check this api in postman to check :

- Server accepts correct request
- Server rejects invalid request
- Response structure is correct
- Performance limits
- Security issues

In Postman : POST https://api.cloudinary.com/v1_1/dnb4odmqv/auto/upload use this as base endpoint

Form-data as body

Key	Value
upload_preset	data_vault_app
folder	data_vault_uploads
public_id	test_file
file	(select file)

API TEST CASE 1 — Valid Upload

Purpose

Verify API uploads file successfully.

Steps in Postman

1. Method → **POST**
2. URL:

https://api.cloudinary.com/v1_1/dnb4odmqv/auto/upload

3. Go to **Body** → **form-data**

Add fields:

Key	Type	Value
upload_preset	Text	data_vault_app
folder	Text	data_vault_uploads
public_id	Text	test_valid_upload
file	File	choose any file

Click **Send**

Expected Result

Response:

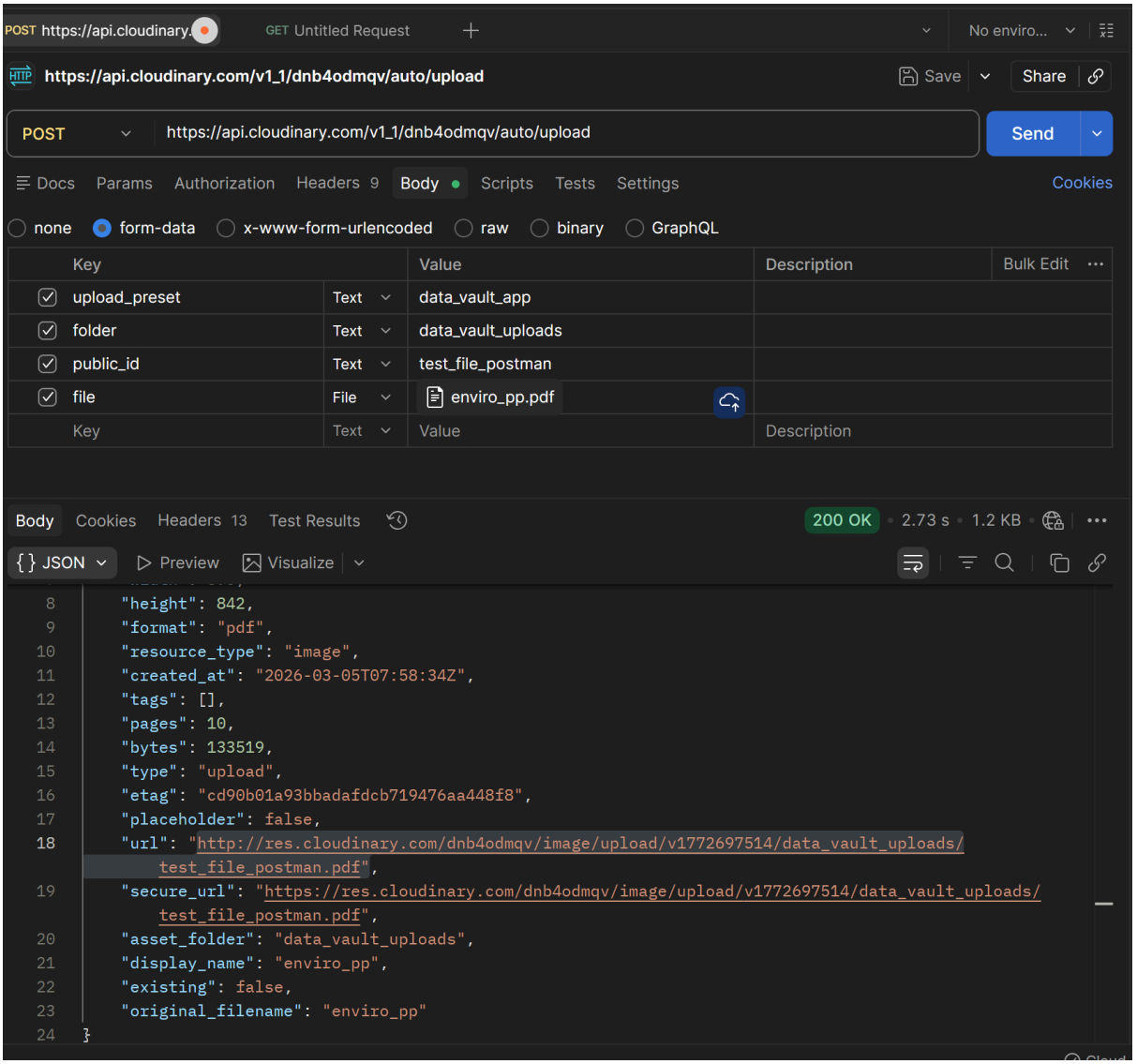
Status: 200 OK

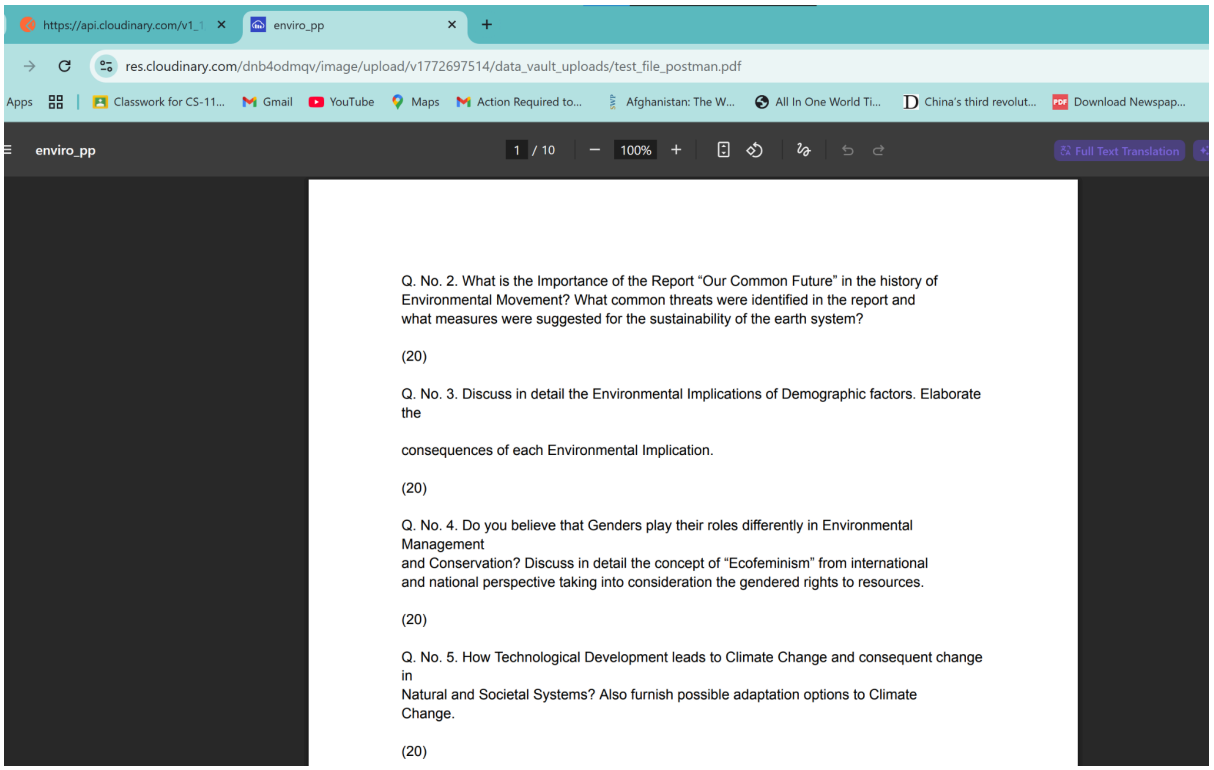
Response body contains:

{

"secure_url": "https://res.cloudinary.com/..."

}





Q. No. 2. What is the Importance of the Report “Our Common Future” in the history of Environmental Movement? What common threats were identified in the report and what measures were suggested for the sustainability of the earth system?

(20)

Q. No. 3. Discuss in detail the Environmental Implications of Demographic factors. Elaborate the

consequences of each Environmental Implication.

(20)

Q. No. 4. Do you believe that Genders play their roles differently in Environmental Management
and Conservation? Discuss in detail the concept of “Ecofeminism” from international and national perspective taking into consideration the gendered rights to resources.

(20)

Q. No. 5. How Technological Development leads to Climate Change and consequent change in
Natural and Societal Systems? Also furnish possible adaptation options to Climate Change.

(20)

Test Case ID: TC-API-01

Title: Verify valid file upload

Tool: Postman

Result: PASS

API TEST CASE 2 — Missing File

Purpose

Verify API rejects request without file.

Steps

In Postman:

1. Keep all parameters same
2. **Remove the "file" row**

Click **Send**

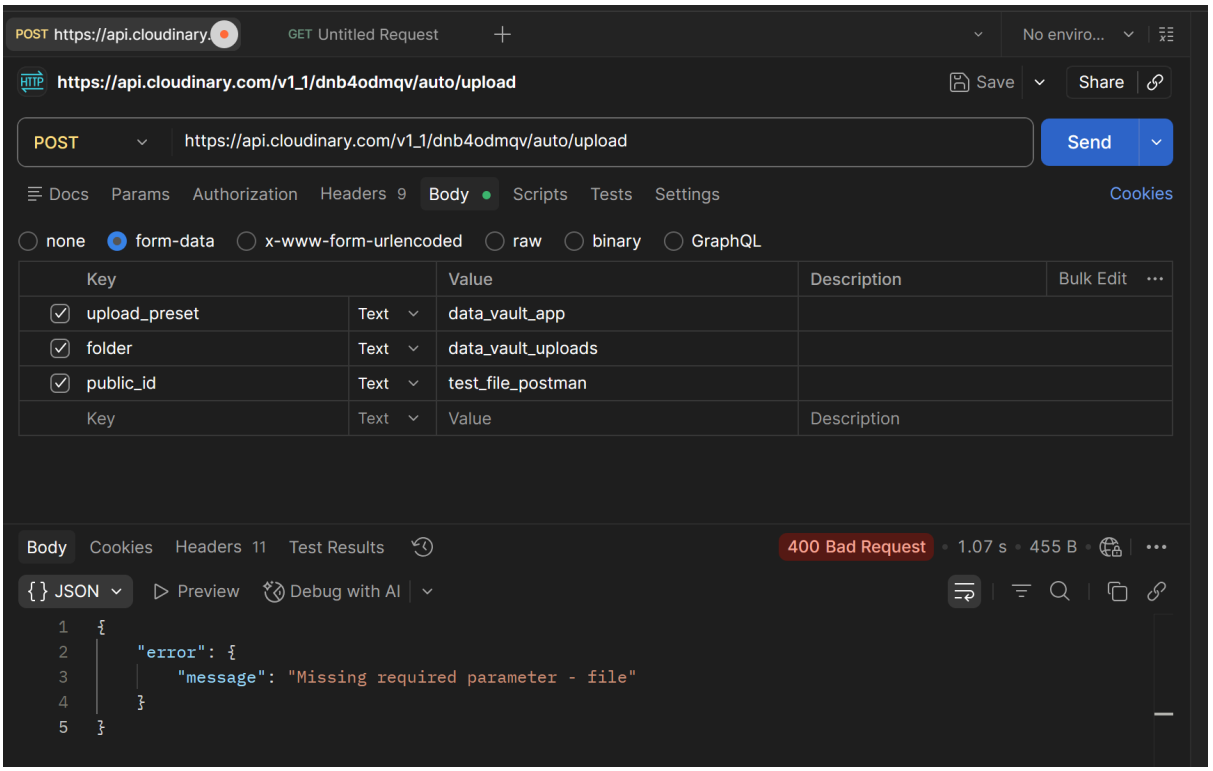
Expected Result

Server should return:

400 Bad Request

Response example:

Missing required parameter - file



Test Case ID: TC-API-02

Title: Verify upload fails without file

Result: PASS

API TEST CASE 3 — Wrong Upload Preset

Purpose

Verify API rejects invalid configuration.

Steps

Change:

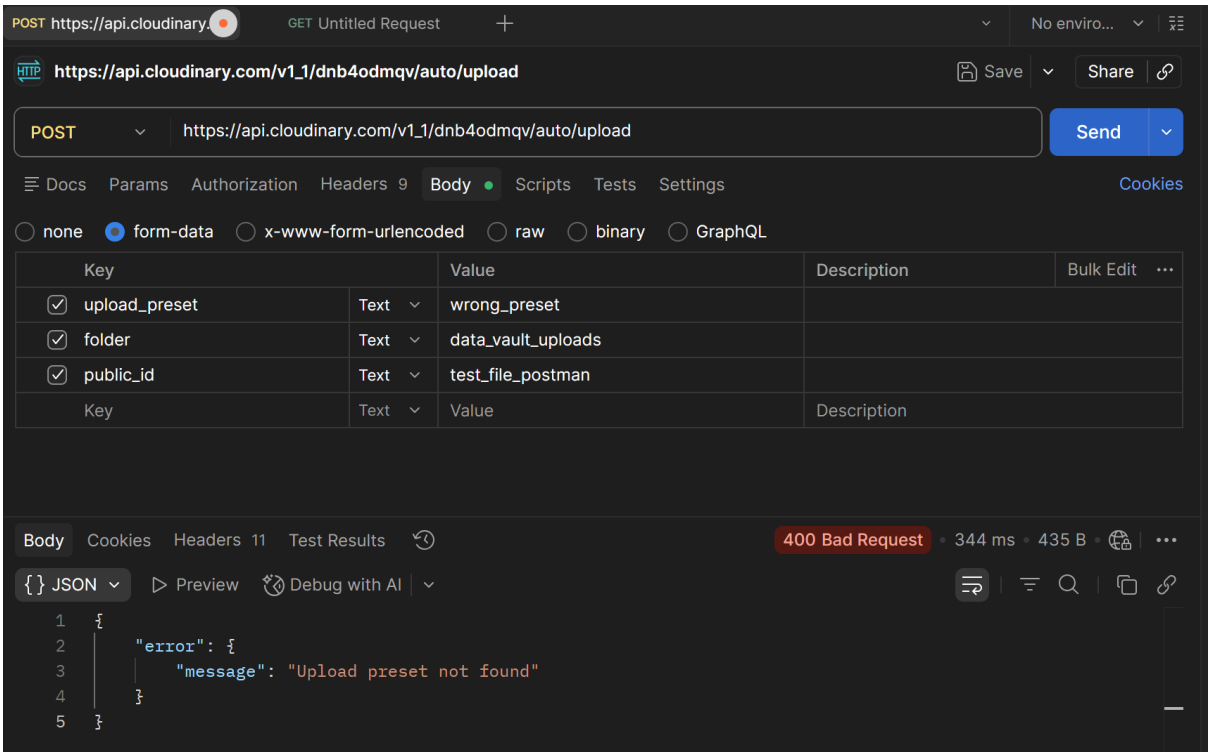
upload_preset = wrong_preset

Keep other parameters the same.

Click **Send**

Expected Result

Response: 400 error



Test Case ID: TC-API-03

Title: Verify API rejects invalid upload preset

Result: PASS

Test Case	Description	Status
TC-API-01	Valid file upload	PASS
TC-API-02	Upload without file	PASS
TC-API-03	Invalid preset	PASS

QA Testing Summary

This project involved performing comprehensive software testing on a file sharing and upload system. The objective was to verify the system's functionality, performance, and API behavior.

Testing Types Performed

- Functional Testing
- Non-Functional Testing (Performance Testing)
- API Testing using Postman
- Bug Reporting and Tracking.

Tools Used

- Jira (Bug Tracking)
- Postman (API Testing)
- Manual Testing Techniques

Conclusion

The testing process successfully identified critical functional and performance issues within the system. The defects discovered highlight areas where the application requires improvements in concurrency control and file upload handling. Overall, the testing demonstrated the importance of structured test cases, API validation, and proper bug reporting in ensuring software quality.